
**INSTITUT SUPERIEUR D'INFORMATIQUE
ET DES TECHNOLOGIES DE COMMUNICATION
DE SOUSSE**



Rapport de stage de fin d'études

Présenté en vue de l'obtention du diplôme de Licence en Science de
l'Informatique (Computer Science)

Spécialité : Informatique et Multimédia

**Souk-AI : Plateforme de Commerce Social Intelligent
pour la Tunisie**

Réalisé par :
Youssef Mestiri

Encadrant académique : Mme. Hanen Zorgati

Encadrant professionnel : Mr. Hamza Sayari

Société d'accueil
Web Media International SARL



Année universitaire : 2025/2026

Dédicace

Je consacre ce modeste travail :

À ma mère, **Sonia Nagbou,**

toi qui es le fil conducteur de ma vie et l'éclat de chaque jour. Ta tendresse, ta générosité et tes innombrables sacrifices ont façonné la personne que je suis. Merci pour ton amour infini et pour tous ces instants où ta présence m'a porté.

À mon père, **Habib Mestiri,**

ton exemple, ta confiance et tes conseils avisés ont été pour moi autant de repères essentiels. Je t'exprime toute ma gratitude et mon affection pour le soutien constant que tu m'as offert, et pour la force que tu m'as insufflée dans tous mes projets.

Remerciements

Je souhaite exprimer ma profonde gratitude à mon encadrant académique, Mme. Hanen Zorgati. Merci pour votre disponibilité constante, vos conseils clairs et votre expertise, qui m'ont guidé à chaque étape de l'élaboration de ce mémoire. Vos remarques rigoureuses ont grandement contribué à enrichir la qualité de ce travail.

Je tiens également à remercier chaleureusement mon encadrante en entreprise, Mr, Hamza Sayari. Votre soutien professionnel, vos retours précis et votre bienveillance ont été essentiels pour concrétiser ce projet. Grâce à vos recommandations, j'ai pu mieux appréhender les enjeux pratiques et ajuster mes méthodes de travail au contexte réel.

Je remercie sincèrement les membres du jury pour le temps précieux qu'ils ont consacré à la lecture attentive de ce travail. Leurs remarques constructives et leur regard critique ont permis d'améliorer la pertinence de mes analyses et la clarté de la présentation.

Enfin, je dédie une mention toute particulière à ma famille pour son soutien indéfectible. Merci à mes parents et à mes proches pour leur confiance, leurs encouragements et leur patience. Votre présence rassurante m'a donné la motivation nécessaire pour mener à bien ce projet, même dans les moments les plus exigeants.

Table des matières

Dédicace	1
Remerciements	2
Introduction générale	7
1 Présentation de l'entreprise et contexte général	8
Sommaire	8
1.1 Introduction	9
1.2 Présentation de la société	9
1.2.1 Implantation géographique et spécialisation	9
1.2.2 Activités et organisation	9
1.3 Analyse fonctionnelle	10
1.3.1 Problématique	10
1.3.2 Solution proposée	10
1.3.3 Objectif de l'application	11
1.4 Conclusion	12
2 Analyse et spécification des besoins	13
Sommaire	13
2.1 Introduction	14
2.2 Capture des besoins	14
2.2.1 Identification des acteurs	14
2.2.2 Spécification des besoins fonctionnels	14
2.2.3 Besoins non fonctionnels	16
2.3 Architecture et environnements de développement	17
2.3.1 Architecture du projet	17
2.3.2 Environnement matériel	21
2.3.3 Environnement logiciel	21
2.4 Conclusion	24
3 Étude conceptuelle	25
Sommaire	25
3.1 Introduction	26
3.2 Méthodes et outils de modélisation	26

3.2.1	Langage de modélisation (UML)	26
3.2.2	L'outil de modélisation	26
3.3	Diagrammes de cas d'utilisation	26
3.3.1	Diagramme de cas d'utilisation global	26
3.3.2	Diagramme de cas d'utilisation de l'administrateur	29
3.3.3	Diagramme de cas d'utilisation de la boutique	29
3.3.4	Diagramme de cas d'utilisation du client	30
3.4	Étude de quelques diagrammes des séquences	35
3.4.1	Diagramme de séquence : recherche sémantique de produits . . .	35
3.4.2	Diagramme de séquence : passer une commande	36
3.4.3	Diagramme de séquence : modification du statut d'une commande	37
3.4.4	Diagramme de séquence : reformulation d'un produit par IA . .	38
3.5	Diagramme de classes	40
3.6	Conclusion du chapitre	42
4	Réalisation de l'application	43
	Conclusion Générale	44
	Bibliographie	45

Table des figures

2.1	Architecture générale de Souk AI	17
2.2	Architecture MVC du back-end Laravel de Souk AI	18
2.3	Visual Studio Code	22
2.4	Laravel	22
2.5	React	22
2.6	Tailwind CSS	23
2.7	MySQL	23
2.8	Docker	23
2.9	Postman	24
3.1	Diagramme de cas d'utilisation global	28
3.2	Diagramme de cas d'utilisation de l'administrateur	29
3.3	Diagramme de cas d'utilisation de la boutique	30
3.4	Diagramme de cas d'utilisation du client	30
3.5	Diagramme de séquence de la recherche sémantique de produits	35
3.6	Diagramme de séquence du passage de commande	36
3.7	Diagramme de séquence de la modification du statut d'une commande	37
3.8	Diagramme de séquence de la reformulation d'un produit par IA	38
3.9	Diagramme de classes	40

Liste des tableaux

1.1	Informations générales de l'entreprise	9
3.1	Description du cas d'utilisation « Rechercher un produit avec l'IA » . .	31
3.2	Description du cas d'utilisation « Gérer les produits et leurs variantes »	32
3.3	Description du cas d'utilisation « Passer une commande »	33
3.4	Description du cas d'utilisation « Gérer les utilisateurs et les boutiques »	34

Introduction générale

Le commerce électronique connaît, en Tunisie comme ailleurs, une transformation portée par l'essor des réseaux sociaux et l'intégration croissante de l'intelligence artificielle dans les parcours d'achat. Les consommateurs attendent désormais des plateformes capables de comprendre leurs besoins en langage naturel et de leur offrir une expérience d'achat fluide et personnalisée, tandis que les vendeurs cherchent des outils simples pour toucher une audience plus large.

C'est dans ce contexte que s'inscrit Souk AI (EcoMarket Connect), une marketplace de commerce social propulsée par l'intelligence artificielle, conçue pour le marché tunisien. La plateforme met en relation quatre types d'acteurs : les boutiques, qui proposent leurs produits ; les clients, qui les recherchent et les achètent ; les influenceurs, qui participent à leur promotion moyennant commission ; et les sociétés de transport et leurs livreurs, chargés de l'acheminement des commandes. Un administrateur supervise l'ensemble de cet écosystème.

Souk AI se distingue par plusieurs innovations : une recherche sémantique de produits basée sur les embeddings de Gemini, permettant une recherche en langage naturel ; une traduction automatique (Gemini/Grok) assurant un contenu trilingue (FR/AR/EN) sans effort de saisie répétée ; une gestion récursive des variantes de produits sous forme d'arbre, avec SKU partagés entre plusieurs branches ; un système de commissions pour les influenceurs ; et une livraison organisée par zones géographiques, regroupant les gouvernorats tunisiens.

Techniquement, la plateforme repose sur un back-end Laravel 12 exposant une API sécurisée par Sanctum, un front-end combinant React (tableaux de bord) et Blade (vitrine publique optimisée SEO) avec Tailwind CSS, une base MySQL structurée en UUID, et une infrastructure Docker garantissant un environnement reproductible.

Ce rapport présente la démarche suivie pour la conception de Souk AI : le chapitre 2 spécifie les besoins fonctionnels et non fonctionnels, le chapitre 3 détaille l'étude conceptuelle (diagrammes UML), et les chapitres suivants abordent la mise en œuvre et les résultats obtenus.

Chapitre 1

Présentation de l'entreprise et contexte général

Sommaire

- 1.1 Introduction
- 1.2 Présentation de la société
 - 1.2.1 Implantation géographique et spécialisation
 - 1.2.2 Activités et organisation
- 1.3 Analyse fonctionnelle
 - 1.3.1 Problématique
 - 1.3.2 Solution proposée
 - 1.3.3 Objectif de l'application
 - 1.3.4 Aperçu technologique de la solution
- 1.4 Conclusion

1.1 Introduction

Dans ce chapitre, nous présentons tout d'abord l'entreprise Web Media International, au sein de laquelle le projet de fin d'études a été réalisé. Nous introduisons ensuite le contexte du projet Souk AI, en mettant en évidence la problématique à laquelle il répond, la solution proposée ainsi que les principaux objectifs de la plateforme.

1.2 Présentation de la société

Web Media International est une agence web basée à Tunis qui accompagne les entreprises dans leur transformation digitale. Elle intervient notamment dans la création de sites internet, le développement web et mobile sur mesure, la conception de marketplaces et de plateformes web, l'UX/UI design, le référencement SEO/SEA, le marketing digital ainsi que l'intégration de solutions d'intelligence artificielle et d'automatisation.

1.2.1 Implantation géographique et spécialisation

Élément	Valeur
Raison sociale	Web Media International SARL
Adresse	66 Avenue Jean Jaurès, Centre Mariem, étage 3, Bureau N°1, 1001 Tunis, Tunisie
Activité	Développement web et mobile, marketplaces, plateformes sur mesure, IA, UX/UI, SEO et marketing digital
Forme juridique	SARL
RNE	1235820P
Site web	webmedia-tunisie.com
Téléphone	+216 25 472 525 / +216 29 888 079
E-mail	commercial@webmediatunisie.com

TABLE 1.1 – Informations générales de l'entreprise

1.2.2 Activités et organisation

Web Media propose une offre digitale couvrant notamment les domaines suivants :

- Création de sites web : sites vitrines, e-commerce, WordPress et solutions professionnelles.
- Développement d'applications web et mobiles sur mesure.

- Conception et développement de marketplaces, plateformes web et tableaux de bord.
- Intelligence artificielle : chatbots, agents IA, automatisation et optimisation de l'expérience utilisateur.
- UX/UI design et conception graphique.
- Référencement naturel et payant (SEO/SEA), marketing digital et génération de leads.
- Maintenance, sécurité et optimisation continue des solutions digitales.

L'agence indique également s'appuyer sur une méthodologie de travail structurée autour de l'analyse et du cadrage des besoins, de la conception, du développement, du déploiement et de l'accompagnement continu. Cette expertise constitue un environnement adapté au développement de Souk AI, qui combine les problématiques d'une marketplace avec l'intégration de fonctionnalités reposant sur l'intelligence artificielle.

1.3 Analyse fonctionnelle

1.3.1 Problématique

Le commerce en ligne permet aujourd'hui aux utilisateurs d'accéder à un nombre très important de produits et de boutiques. Cependant, cette diversité peut rendre la recherche d'un produit plus longue et moins intuitive lorsque l'utilisateur doit parcourir plusieurs boutiques, catégories et fiches produits avant de trouver une offre correspondant réellement à son besoin.

Dans le contexte tunisien, le projet Souk AI part ainsi du besoin de proposer un espace centralisé regroupant plusieurs boutiques, tout en facilitant la découverte des produits. Une recherche classique basée uniquement sur des mots-clés ou des catégories peut être insuffisante lorsque l'utilisateur formule son besoin de manière naturelle, imprécise ou selon plusieurs critères.

La problématique du projet peut donc être formulée comme suit : comment concevoir une marketplace multi-boutiques en Tunisie capable de centraliser une offre diversifiée tout en utilisant l'intelligence artificielle pour rendre la recherche de produits plus simple, pertinente et intuitive ?

1.3.2 Solution proposée

Pour répondre à cette problématique, nous proposons Souk AI, une marketplace web destinée au marché tunisien. La plateforme permet de regrouper plusieurs boutiques et leurs produits au sein d'un même environnement numérique, afin d'offrir à l'utilisateur un point d'accès unique à une offre diversifiée.

L'une des principales particularités de Souk AI réside dans l'intégration de fonctionnalités d'intelligence artificielle dédiées à la recherche. L'objectif est de permettre à l'utilisateur d'exprimer plus naturellement ce qu'il recherche et de l'aider à identifier plus rapidement les produits correspondant à son besoin, au-delà d'une navigation classique par catégories.

La solution vise notamment à permettre aux utilisateurs de :

- Consulter les produits proposés par plusieurs boutiques depuis une plateforme unique.
- Rechercher et filtrer plus facilement les produits disponibles.
- Utiliser des fonctionnalités de recherche assistées par intelligence artificielle afin d'obtenir des résultats plus pertinents.
- Consulter les informations relatives aux produits et aux boutiques.
- Profiter d'une expérience de navigation simple, fluide et adaptée à une marketplace.

1.3.3 Objectif de l'application

L'objectif principal de Souk AI est de développer une marketplace moderne adaptée au marché tunisien, capable de mettre en relation les utilisateurs avec plusieurs boutiques et de faciliter la découverte de produits grâce à l'intelligence artificielle.

Plus précisément, l'application vise à :

- Centraliser sur une même plateforme les offres provenant de plusieurs boutiques.
- Faciliter l'accès des utilisateurs à un catalogue de produits diversifié.
- Améliorer la recherche de produits grâce à des fonctionnalités basées sur l'intelligence artificielle.
- Réduire le temps nécessaire pour trouver un produit correspondant aux critères de l'utilisateur.
- Offrir aux boutiques un espace leur permettant de présenter et gérer leurs produits.
- Mettre à disposition de l'administrateur les fonctionnalités nécessaires à la gestion et au suivi de la plateforme.

Le projet peut ainsi être structuré autour de trois espaces fonctionnels principaux :

- Module Administrateur
- Module Boutique / Vendeur
- Module Client / Utilisateur

1.4 Conclusion

Dans ce chapitre, nous avons présenté Web Media International ainsi que son domaine d'activité. Nous avons ensuite introduit le contexte du projet Souk AI, la problématique liée à la centralisation de plusieurs boutiques et à l'amélioration de la recherche de produits, puis la solution proposée et ses principaux objectifs. Le chapitre suivant sera consacré à l'analyse et à la spécification détaillée des besoins fonctionnels et non fonctionnels de la plateforme.

Sources institutionnelles utilisées pour les informations relatives à Web Media : site officiel Web Media Tunisie (pages Contact, Services, Développement web et Mentions légales).

Chapitre 2

Analyse et spécification des besoins

Sommaire

- 2.1 Introduction
- 2.2 Capture des besoins
 - 2.2.1 Identification des acteurs
 - 2.2.2 Spécification des besoins fonctionnels
 - 2.2.3 Besoins non fonctionnels
- 2.3 Architecture et environnements de développement
 - 2.3.1 Architecture du projet
 - 2.3.2 Environnement matériel
 - 2.3.3 Environnement logiciel
- 2.4 Conclusion

2.1 Introduction

Dans ce chapitre, nous allons identifier et spécifier les différents besoins fonctionnels et non fonctionnels de notre application Souk AI. Nous commencerons par présenter les principaux acteurs de la plateforme ainsi que les fonctionnalités qui leur sont associées. Ensuite, nous présenterons l'architecture adoptée pour le développement de l'application. Enfin, nous décrirons les environnements matériels et logiciels utilisés pour la réalisation du projet.

2.2 Capture des besoins

Cette section est consacrée à l'identification des différents acteurs qui interagissent avec Souk AI, ainsi qu'à la spécification des besoins fonctionnels et non fonctionnels de la plateforme.

2.2.1 Identification des acteurs

Souk AI est une marketplace permettant de regrouper plusieurs boutiques et leurs produits au sein d'une même plateforme. Les principaux acteurs qui interagissent avec l'application sont les suivants :

- **Le visiteur** : utilisateur non authentifié qui peut accéder à la plateforme, consulter le catalogue, rechercher des produits et créer un compte ou se connecter.
- **Le client** : utilisateur authentifié qui peut rechercher des produits, utiliser la recherche intelligente basée sur l'intelligence artificielle, gérer son panier, passer des commandes, suivre ses commandes et gérer son profil.
- **La boutique** : utilisateur professionnel de la plateforme qui dispose d'un espace dédié lui permettant de gérer sa boutique, ses produits et de consulter les commandes associées.
- **L'administrateur** : utilisateur disposant des droits nécessaires pour administrer la plateforme. Il assure notamment la gestion des utilisateurs, des boutiques, des catégories et des commandes.

Le client constitue une extension du visiteur : après authentification, il bénéficie des fonctionnalités accessibles au visiteur ainsi que des fonctionnalités spécifiques liées à son compte et à ses achats.

2.2.2 Spécification des besoins fonctionnels

Les besoins fonctionnels représentent l'ensemble des fonctionnalités que l'application doit mettre à disposition de ses différents acteurs.

Spécifications fonctionnelles du module Visiteur L'application doit permettre au visiteur de :

- Consulter le catalogue des produits disponibles sur la plateforme.
- Parcourir les produits proposés par les différentes boutiques.
- Rechercher un produit.
- Consulter les informations et les détails d'un produit.
- Créer un compte.
- Se connecter à la plateforme.

Spécifications fonctionnelles du module Client Après authentification, l'application doit permettre au client de :

- Gérer ses informations personnelles et son profil.
- Consulter le catalogue et rechercher des produits.
- Utiliser la recherche intelligente basée sur l'intelligence artificielle afin d'obtenir des résultats correspondant plus précisément à son besoin.
- Ajouter ou supprimer des produits de son panier.
- Modifier les quantités des produits présents dans son panier.
- Passer une commande.
- Consulter et suivre ses commandes.
- Consulter le statut de ses commandes.
- Noter et commenter les produits.

Spécifications fonctionnelles du module Boutique L'application doit permettre à la boutique de :

- Accéder à son espace après authentification.
- Consulter et gérer les informations relatives à sa boutique.
- Ajouter de nouveaux produits.
- Modifier les informations relatives à ses produits.
- Supprimer des produits.
- Gérer le prix, le stock, les promotions et l'état des produits.
- Ajouter et gérer les images associées aux produits.
- Consulter les commandes relatives à ses produits.

Spécifications fonctionnelles du module Administrateur L'administrateur assure la gestion globale de Souk AI. L'application doit lui permettre de :

- S'authentifier et accéder à son espace d'administration.
- Consulter et gérer les utilisateurs de la plateforme.
- Bloquer ou débloquer un utilisateur.
- Créer et gérer les boutiques.
- Ajouter, modifier ou désactiver une boutique.
- Créer et gérer les catégories de produits.
- Organiser les catégories et sous-catégories.
- Consulter les commandes effectuées sur la plateforme.
- Modifier le statut d'une commande.
- Superviser les produits et les différentes activités de la marketplace.

2.2.3 Besoins non fonctionnels

En plus des fonctionnalités précédemment décrites, Souk AI doit respecter un ensemble de besoins non fonctionnels permettant d'assurer la qualité, la sécurité et la fiabilité de la plateforme.

- **Performance** : l'application doit assurer des temps de réponse réduits et permettre une navigation fluide, notamment lors de la consultation du catalogue et de la recherche de produits.
- **Sécurité** : l'accès aux espaces privés doit être protégé par un mécanisme d'authentification. Les droits d'accès doivent également être adaptés au rôle de chaque utilisateur.
- **Disponibilité** : la plateforme doit être accessible aux utilisateurs afin qu'ils puissent consulter les produits et accéder aux services proposés à tout moment.
- **Ergonomie** : les différentes interfaces doivent être simples, intuitives et cohérentes afin de faciliter l'utilisation de la plateforme.
- **Responsive design** : l'interface doit s'adapter aux différentes tailles d'écran afin de garantir une expérience satisfaisante sur ordinateur, tablette et smartphone.
- **Maintenabilité** : le code de l'application doit être organisé de manière structurée afin de faciliter sa maintenance et son évolution.
- **Évolutivité** : l'architecture de Souk AI doit permettre l'ajout futur de nouvelles fonctionnalités, de nouvelles boutiques et de nouveaux services.
- **Fiabilité** : les données relatives aux utilisateurs, produits, boutiques et commandes doivent être enregistrées et traitées de manière cohérente.

2.3 Architecture et environnements de développement

Dans cette section, nous présentons l'architecture générale adoptée pour Souk AI, ainsi que les principaux environnements matériels et logiciels utilisés pour son développement.

2.3.1 Architecture du projet

L'application Souk AI repose sur une architecture séparant la partie front-end, responsable de l'interface utilisateur, de la partie back-end, responsable de la logique métier, du traitement des requêtes et de la gestion des données.

La figure 2.1 présente l'architecture générale de la plateforme ainsi que les principaux échanges entre ses différentes composantes.

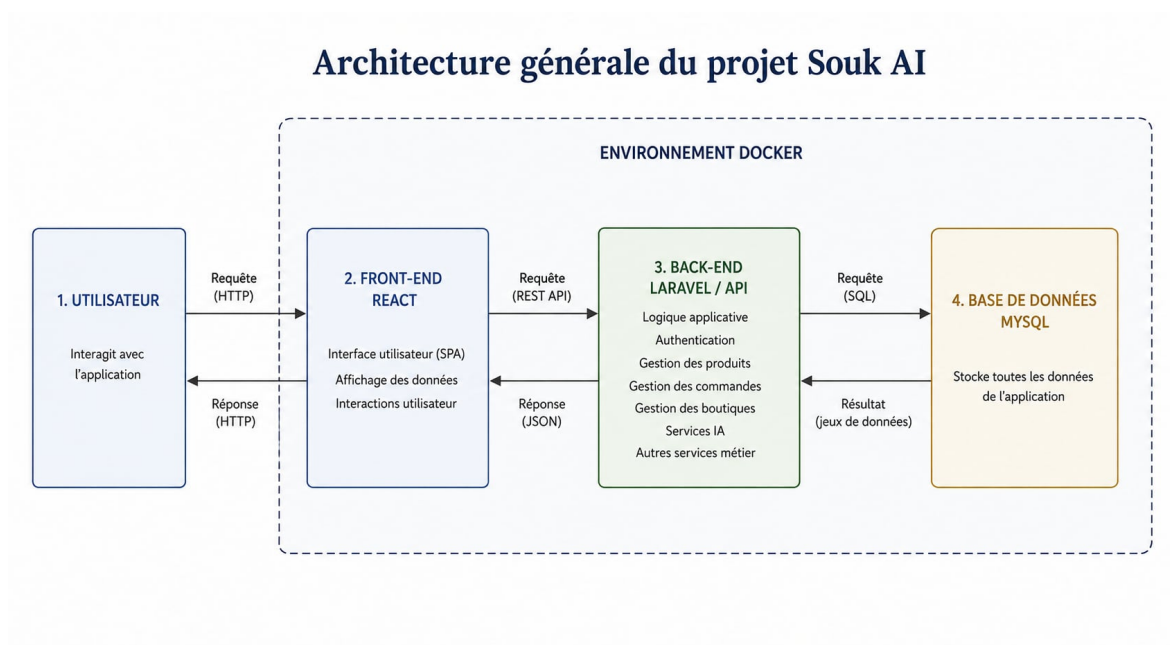


FIGURE 2.1 – Architecture générale de Souk AI

La partie front-end est développée avec **React**. Elle représente l'interface avec laquelle les différents utilisateurs interagissent. Elle permet notamment l'affichage du catalogue, la recherche de produits, la gestion du panier ainsi que l'accès aux différents espaces de la plateforme.

La partie back-end est développée avec **Laravel**. Elle assure le traitement de la logique métier de l'application, la gestion des utilisateurs, des boutiques, des produits, des catégories et des commandes. Elle assure également la communication avec la base de données.

La communication entre le front-end et le back-end est réalisée à travers des **API**. React envoie des requêtes au serveur Laravel qui traite ces requêtes, applique les règles

métier nécessaires et retourne les données demandées.

Les informations de la plateforme sont stockées dans une base de données **MySQL**. Celle-ci assure notamment la persistance des données relatives aux utilisateurs, boutiques, catégories, produits, images et commandes.

L'environnement de l'application est également conteneurisé avec **Docker**, permettant d'isoler les différents services nécessaires au projet et de disposer d'un environnement de développement cohérent et reproductible.

L'architecture générale peut ainsi être résumée par le flux suivant :

Utilisateur → Interface React → API Laravel → MySQL

Cette séparation permet d'améliorer la modularité de l'application et de faciliter sa maintenance et son évolution.

Architecture MVC du back-end

Le back-end de Souk AI, développé avec Laravel, adopte également le pattern architectural **MVC (Model–View–Controller)**. Cette organisation permet de séparer les responsabilités liées à la gestion des données, au traitement des requêtes et à la présentation des informations.

La figure 2.2 présente l'organisation simplifiée du back-end selon le pattern MVC.

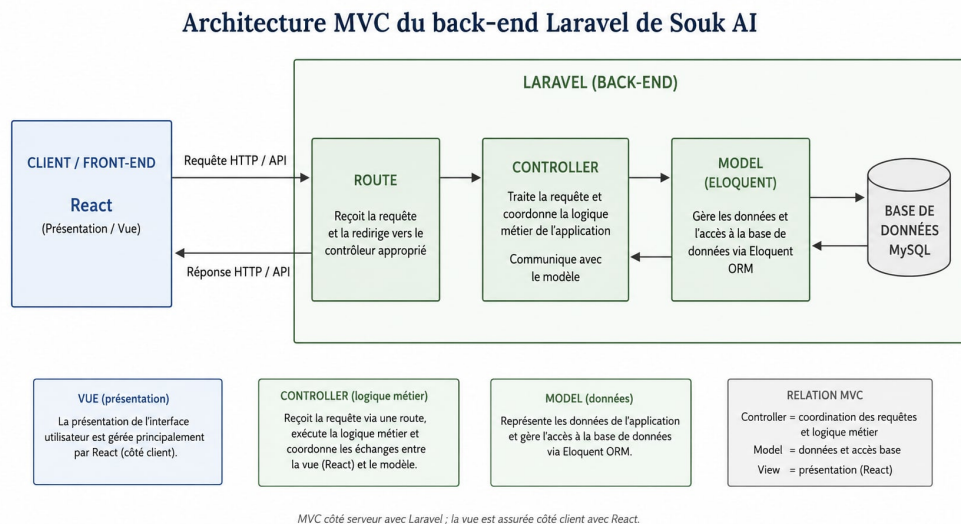


FIGURE 2.2 – Architecture MVC du back-end Laravel de Souk AI

Dans le contexte de Souk AI, le fonctionnement peut être résumé comme suit :

Requête React → Route Laravel → Controller → Model → MySQL

Le résultat du traitement est ensuite retourné au front-end sous la forme d'une réponse API.

Model Le **Model** représente les données et les entités métier de l'application. Dans Laravel, les modèles sont principalement utilisés avec Eloquent ORM afin de faciliter les opérations avec la base de données.

Dans Souk AI, les modèles correspondent notamment aux principales entités de la plateforme, telles que :

- les utilisateurs ;
- les boutiques ;
- les catégories ;
- les produits ;
- les variantes de produits ;
- les commandes ;
- les lignes de commande.

Les modèles permettent également de définir les relations entre ces différentes entités et de manipuler les données stockées dans MySQL.

Controller Le **Controller** constitue le point de traitement des requêtes reçues par l'application. Il reçoit les requêtes provenant notamment du front-end React, applique le traitement nécessaire et fait appel aux modèles lorsque des opérations sur les données sont nécessaires.

Dans Souk AI, les controllers prennent notamment en charge les opérations liées à :

- la gestion des utilisateurs ;
- la gestion des boutiques et des produits ;
- la gestion des catégories ;
- la gestion du panier et des commandes ;
- la recherche de produits ;
- le traitement des requêtes envoyées par les interfaces React.

Après traitement, le controller retourne généralement une réponse structurée destinée au front-end à travers l'API.

View Dans le pattern MVC classique, la **View** représente la couche responsable de la présentation des données à l'utilisateur.

Dans Souk AI, cette responsabilité est répartie selon le type d'interface. Les interfaces dynamiques du tableau de bord sont développées avec **React**, tandis que les pages publiques utilisent également les templates **Blade** de Laravel.

Ainsi, React constitue principalement la couche de présentation du tableau de bord et communique avec le back-end Laravel à travers les API. Les templates Blade sont utilisés pour certaines interfaces publiques intégrées directement à l'application Laravel.

Flux de traitement d'une requête Le traitement d'une opération peut être illustré par le scénario suivant : lorsqu'un utilisateur effectue une action depuis l'interface React, une requête est envoyée vers une route de l'API Laravel. La route dirige ensuite la requête vers le controller approprié. Celui-ci effectue les traitements nécessaires et peut interagir avec les modèles Eloquent afin de consulter ou modifier les données dans MySQL.

Une fois le traitement terminé, Laravel génère une réponse contenant les informations demandées. Cette réponse est ensuite transmise à React, qui met à jour l'interface utilisateur.

Cette organisation permet de séparer clairement les responsabilités entre la présentation, le traitement de la logique métier et la gestion des données. Elle facilite ainsi la maintenance du code et permet de faire évoluer indépendamment les différentes parties de la plateforme.

Front-end Le front-end constitue la partie visible de l'application. Développé avec React, il assure l'affichage des différentes interfaces et la gestion des interactions avec les utilisateurs.

L'utilisation de **Tailwind CSS** permet de gérer la présentation graphique et la mise en page des différentes interfaces. Elle facilite également la création d'une interface responsive adaptée aux différentes tailles d'écran.

Le front-end communique avec le back-end à travers les API Laravel afin de récupérer ou transmettre les données nécessaires au fonctionnement de la plateforme.

Back-end Le back-end de Souk AI est développé avec Laravel. Il représente la partie serveur de l'application et prend en charge la logique métier.

Il assure notamment :

- la gestion des utilisateurs et de l'authentification ;
- la gestion des boutiques ;
- la gestion des catégories ;
- la gestion des produits ;
- la gestion des commandes ;
- le traitement des données transmises par le front-end ;
- la communication avec la base de données MySQL ;
- l'intégration des fonctionnalités d'intelligence artificielle.

L'utilisation de Laravel permet ainsi de structurer le back-end autour d'une organisation claire et modulaire, notamment grâce au pattern MVC, à l'ORM Eloquent et au système de routage fourni par le framework.

Base de données La base de données MySQL permet de stocker et d’organiser les différentes informations nécessaires au fonctionnement de Souk AI.

Elle contient notamment les données relatives :

- aux utilisateurs ;
- aux clients ;
- aux boutiques ;
- aux catégories ;
- aux produits ;
- aux variantes de produits ;
- aux images des produits ;
- aux commandes ;
- aux lignes de commande.

Les interactions entre Laravel et MySQL sont principalement réalisées à travers l’ORM Eloquent. Cette approche permet aux modèles Laravel de représenter les principales entités métier et de simplifier les opérations de consultation, d’insertion, de modification et de suppression des données.

2.3.2 Environnement matériel

Pour développer l’application Souk AI, nous avons utilisé un ordinateur disposant des ressources nécessaires à l’exécution simultanée de l’environnement de développement, des différents conteneurs Docker et des services associés au projet. Les principales caractéristiques matérielles de la machine utilisée sont :

- Système d’exploitation : Windows 11.
- Processeur : i5.
- RAM : 16 GB.
- Stockage : 512 GB.

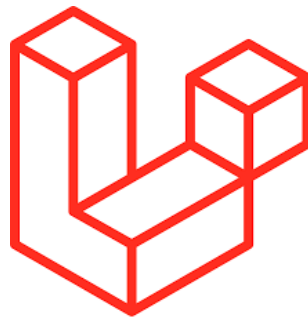
2.3.3 Environnement logiciel

Pour la conception et le développement de Souk AI, plusieurs technologies et outils logiciels ont été utilisés.

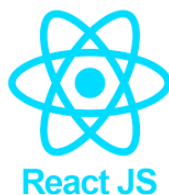
Visual Studio Code Visual Studio Code (VS Code) est l’environnement de développement utilisé pour la réalisation du projet. Il permet d’écrire et d’organiser le code source du front-end et du back-end au sein d’un même environnement. Grâce à son système d’extensions, il facilite également le développement avec les différentes technologies utilisées dans Souk AI [3].

**FIGURE 2.3** – Visual Studio Code

Laravel Laravel est le framework utilisé pour développer la partie back-end de l'application. Basé sur PHP, il fournit une structure adaptée au développement d'applications web et d'API. Dans Souk AI, Laravel est principalement utilisé pour implémenter la logique métier, gérer les différentes ressources de la marketplace, traiter les requêtes provenant du front-end et assurer la communication avec la base de données [5].

**FIGURE 2.4** – Laravel

React React est utilisé pour développer la partie front-end de Souk AI. Cette technologie permet de construire l'interface utilisateur à partir de composants réutilisables et facilite la création d'interfaces dynamiques. React est notamment utilisé pour construire les différentes pages du catalogue, les interfaces liées aux produits, le panier ainsi que les espaces dédiés aux différents utilisateurs [10].

**FIGURE 2.5** – React

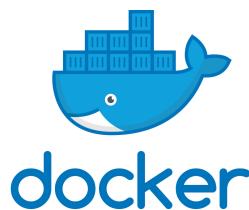
Tailwind CSS Tailwind CSS est utilisé pour la conception graphique des interfaces de Souk AI. Il fournit des classes utilitaires permettant de définir directement la mise en forme des différents composants. Son utilisation facilite la création d'une interface homogène et responsive, tout en permettant d'adapter rapidement l'apparence des différents éléments de la plateforme [13].

**FIGURE 2.6** – Tailwind CSS

MySQL MySQL est le système de gestion de base de données relationnelle utilisé dans le projet. Il permet d'assurer le stockage structuré et persistant des données nécessaires au fonctionnement de la marketplace. Dans Souk AI, MySQL contient notamment les informations relatives aux utilisateurs, aux boutiques, aux catégories, aux produits et aux commandes [15].

**FIGURE 2.7** – MySQL

Docker Docker est utilisé afin de conteneuriser l'environnement de développement de Souk AI. Il permet d'exécuter les différents services nécessaires à l'application dans des environnements isolés et configurés de manière cohérente. Cette approche facilite l'installation du projet, limite les différences de configuration entre les environnements et simplifie le lancement des différents services nécessaires au fonctionnement de l'application [17].

**FIGURE 2.8** – Docker

Postman Postman est un outil qui permet aux développeurs de tester et de documenter les API (Application Programming Interface). Il permet de créer des requêtes HTTP (GET, POST, PUT, etc.), de gérer les paramètres et les entêtes, de vérifier les réponses et les statuts de retour, et de sauvegarder des requêtes pour une utilisation ultérieure. Il permet également de créer des collections de requêtes pour organiser et partager les tests avec d'autres développeurs. Il est disponible en tant qu'application de bureau et en tant qu'extension de navigateur. [20].



FIGURE 2.9 – Postman

2.4 Conclusion

Dans ce chapitre, nous avons identifié les principaux acteurs de Souk AI et défini les besoins fonctionnels et non fonctionnels associés à la plateforme. Nous avons également présenté l'architecture générale du projet, reposant notamment sur React pour le front-end, Laravel pour le back-end et MySQL pour la gestion des données. Enfin, nous avons présenté les principaux outils et technologies utilisés pour le développement, notamment Tailwind CSS, Docker et Visual Studio Code. Le chapitre suivant sera consacré à l'étude conceptuelle de Souk AI, à travers la présentation des outils de modélisation ainsi que des différents diagrammes UML permettant de représenter les acteurs, les fonctionnalités, les interactions et la structure de l'application.

Chapitre 3

Étude conceptuelle

Sommaire

- 3.1 Introduction
- 3.2 Méthodes et outils de modélisation
 - 3.2.1 Langage de modélisation (UML)
 - 3.2.2 L'outil de modélisation
- 3.3 Diagrammes de cas d'utilisation
 - 3.3.1 Diagramme de cas d'utilisation global
 - 3.3.2 Diagramme de cas d'utilisation de l'administrateur
 - 3.3.3 Diagramme de cas d'utilisation de la boutique
 - 3.3.4 Diagramme de cas d'utilisation du client
- 3.4 Étude de quelques diagrammes de séquences
- 3.5 Diagramme de classes
- 3.6 Conclusion du chapitre

3.1 Introduction

Dans ce chapitre, nous présenterons d’abord les outils logiciels et les langages de modélisation utilisés, puis nous détaillerons les diagrammes de cas d’utilisation, les diagrammes de classes et les diagrammes de séquences relatifs à notre application Souk AI.

3.2 Méthodes et outils de modélisation

3.2.1 Langage de modélisation (UML)

Pour concevoir notre système, nous avons opté pour une approche orientée objet, qui constitue une méthode essentielle dans le développement moderne des applications.

Afin de mieux décrire l’architecture du système proposé, nous utiliserons le langage UML, largement reconnu dans le domaine de la modélisation.

UML représente une norme de modélisation orientée objet permettant de décrire et documenter précisément les systèmes d’information.

L’utilisation d’UML répond principalement aux objectifs suivants :

- Structurer formellement la conception de l’application.
- Renforcer la communication entre les différents acteurs d’un projet informatique.
- Assurer la coordination efficace des tâches entre les intervenants.
- Gérer efficacement l’évolution continue du projet informatique.
- Fournir des outils standardisés couvrant divers aspects liés à la conception.

3.2.2 L’outil de modélisation

StarUML est un logiciel de modélisation UML disponible en open source suite à l’arrêt de son exploitation commerciale, sous une version adaptée de la licence GNU GPL. Ce logiciel prend en charge la majorité des diagrammes définis par la norme UML 2.0. Développé en langage Delphi, il intègre également des composants propriétaires issus de cet environnement [?].

3.3 Diagrammes de cas d’utilisation

3.3.1 Diagramme de cas d’utilisation global

Un diagramme global de cas d’utilisation permet d’avoir une vue complète sur l’ensemble du système en illustrant clairement les interactions entre les acteurs et les

différentes fonctionnalités auxquelles ils ont accès. Le diagramme ci-dessous présente les quatre acteurs principaux de Souk AI – Visiteur, Client, Boutique et Administrateur – ainsi que les cas d’utilisation spécifiques associés à chacun, tels que la recherche intelligente de produits, la gestion du panier et des commandes, la gestion des produits et des variantes, ou encore l’administration des utilisateurs et des catégories. Cette représentation simplifiée aide à mieux cerner le rôle de chaque acteur et à définir précisément le périmètre fonctionnel du projet.

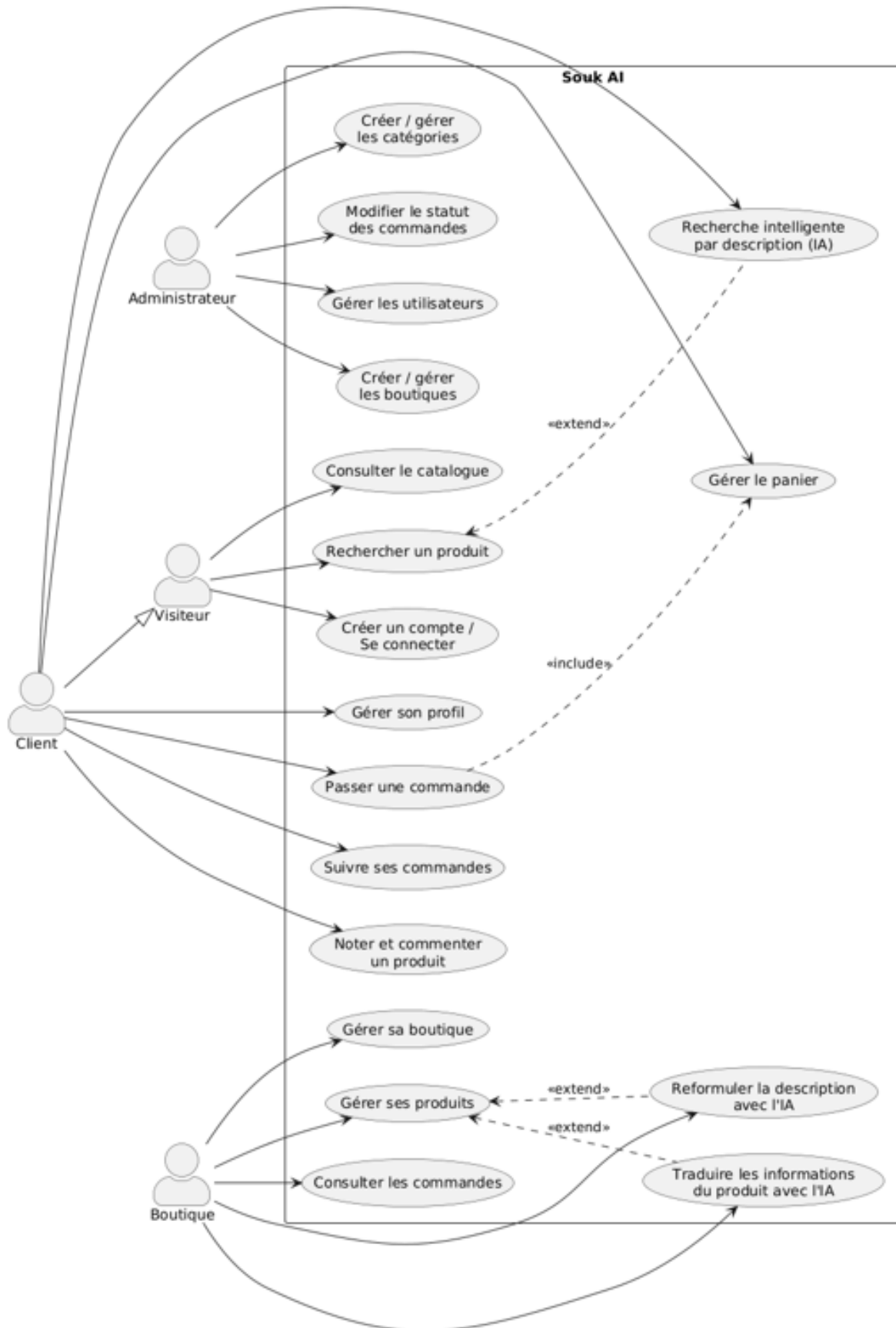


FIGURE 3.1 – Diagramme de cas d'utilisation global

3.3.2 Diagramme de cas d'utilisation de l'administrateur

Ce diagramme de cas d'utilisation illustre les principales interactions entre l'administrateur et le système. Il met en évidence les fonctionnalités permettant à l'administrateur de consulter et de gérer les utilisateurs, les boutiques et les catégories de la plateforme. Ces interactions incluent l'authentification, ainsi que des actions telles que le blocage d'un utilisateur, l'activation ou la désactivation d'une boutique, la création de catégories et sous-catégories, et le suivi des commandes passées sur la marketplace.

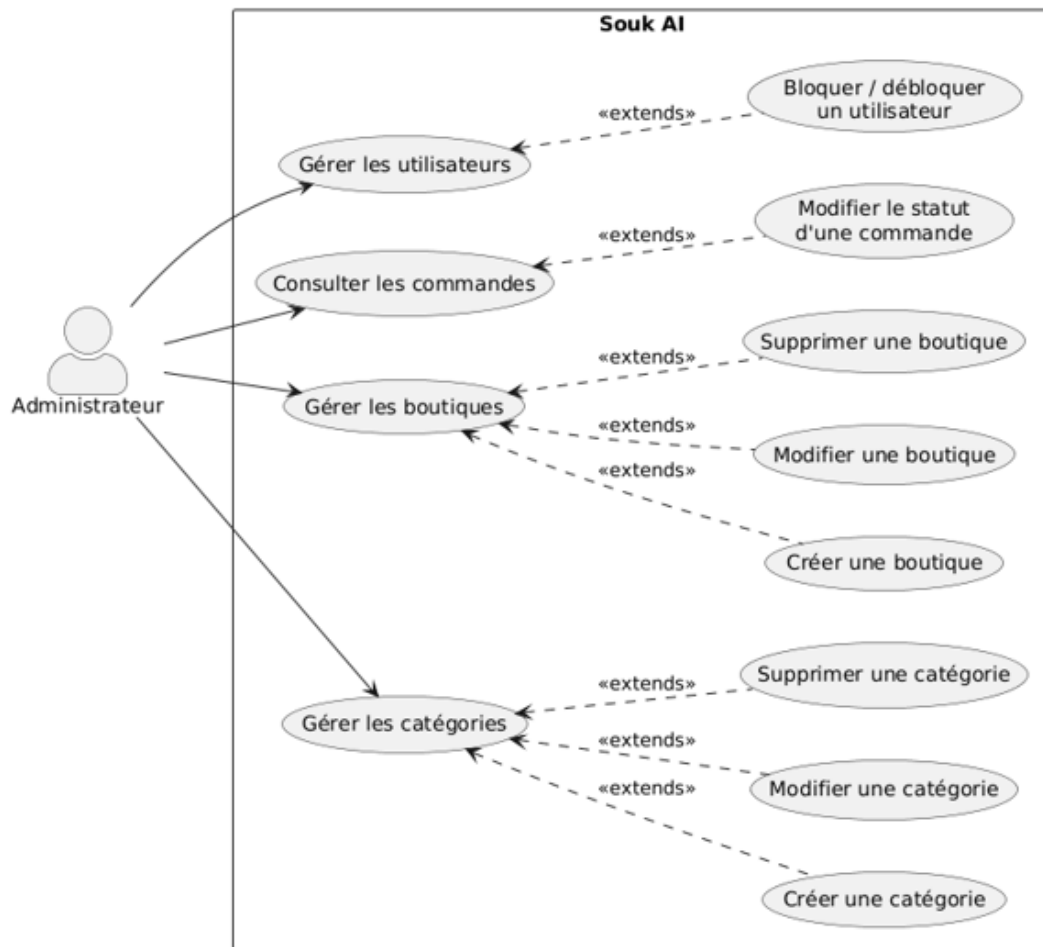


FIGURE 3.2 – Diagramme de cas d'utilisation de l'administrateur

3.3.3 Diagramme de cas d'utilisation de la boutique

Ce diagramme de cas d'utilisation représente les interactions entre la boutique et la plateforme Souk AI. Il illustre les principales actions que la boutique peut effectuer, telles que la gestion de son profil, l'ajout, la modification ou la suppression de produits, la gestion des variantes (couleur, taille, etc.), du stock et des promotions, ainsi que l'ajout d'images associées aux produits. Le diagramme met également en évidence l'authentification nécessaire pour accéder à ces fonctionnalités, ainsi que la consultation des commandes relatives aux produits de la boutique.

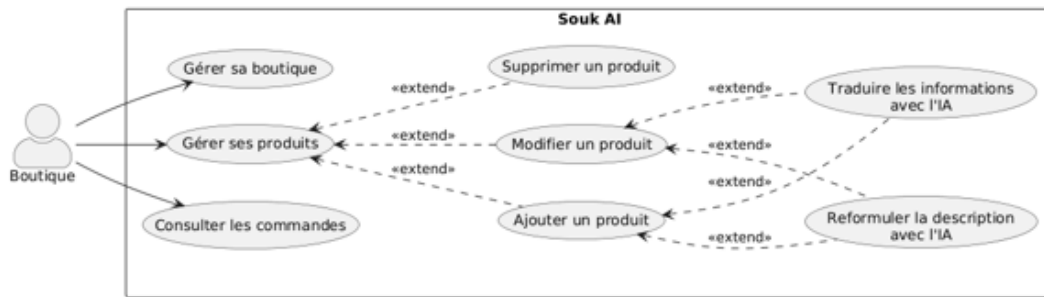


FIGURE 3.3 – Diagramme de cas d'utilisation de la boutique

3.3.4 Diagramme de cas d'utilisation du client

Ce diagramme de cas d'utilisation présente les principales fonctionnalités offertes au client sur la plateforme : de la recherche de produits – classique ou intelligente, basée sur l'intelligence artificielle – à la gestion du panier et au passage de commande, en passant par le suivi du statut des commandes et la possibilité de noter et commenter les produits achetés.

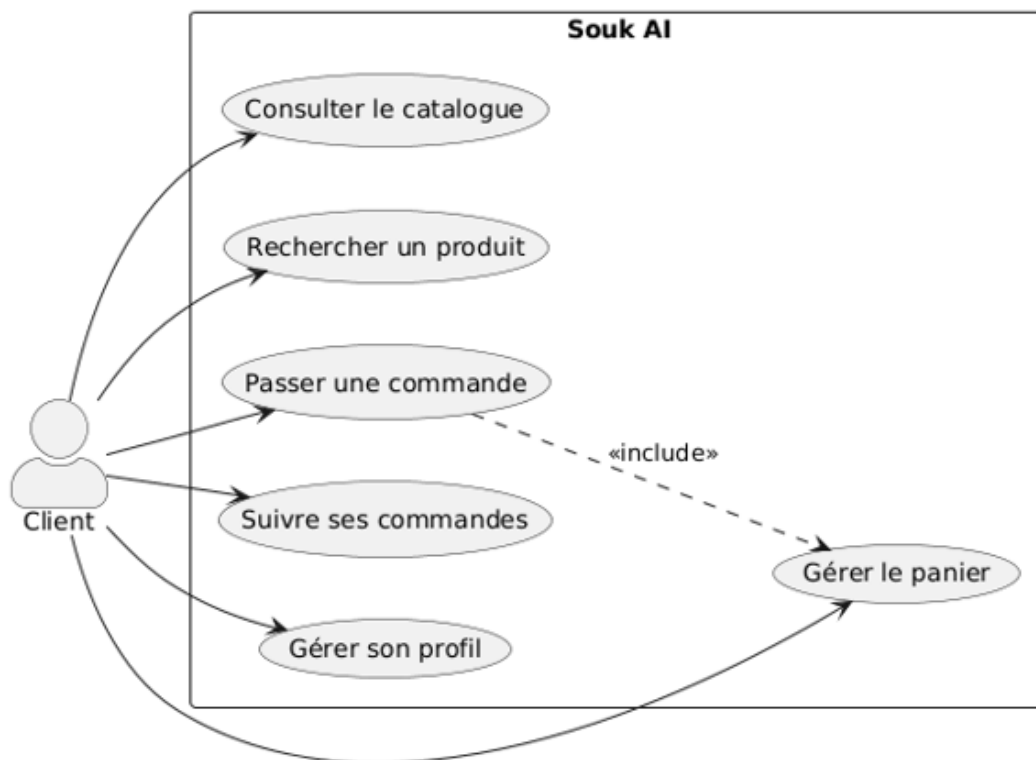


FIGURE 3.4 – Diagramme de cas d'utilisation du client

Description du cas d'utilisation : rechercher un produit avec l'IA

Cas d'utilisation	Rechercher un produit avec l'IA (recherche sémantique)
Résumé	Ce cas permet au client de formuler une recherche en langage naturel et d'obtenir des produits pertinents grâce à la similarité vectorielle des embeddings générés par Gemini.
Acteurs	Client
Scénario nominal	1. Le client accède à la barre de recherche de la plateforme. 2. Il saisit une requête en langage naturel (ex. « chaussures confortables pour marcher »). 3. Le système transmet la requête au <code>ProductSemanticSearchService</code>. 4. Le service génère l'embedding de la requête et le compare aux embeddings des produits déjà calculés et mis en cache (<code>product_search_embeddings</code>). 5. Le système renvoie la liste des produits les plus proches sémantiquement, triés par pertinence.
Scénario d'erreur	1. Le service Gemini est indisponible ou renvoie une erreur. 2. Le système bascule sur une recherche classique par mots-clés et informe le client.
Pré-conditions	Les produits disposent d'un embedding déjà calculé (ou calculé à la volée).
Post-condition	La liste des produits correspondant le mieux à la requête est affichée au client.

TABLE 3.1 – Description du cas d'utilisation « Rechercher un produit avec l'IA »

Description du cas d'utilisation : gérer les produits et leurs variantes

Cas d'utilisation	Gérer les produits et leurs variantes
Résumé	Ce cas permet à la boutique de créer un produit puis de définir son arbre de variantes (couleur, taille, etc.), chaque nœud pouvant porter un SKU, un prix et un stock propres.
Acteurs	Boutique
Scénario nominal	1. La boutique s'authentifie et accède à son espace de gestion des produits. 2. Elle crée un nouveau produit (nom, description, catégorie, prix de base, images). 3. Le système déclenche la traduction automatique EN/FR/AR du contenu via Gemini ou Grok. 4. La boutique ajoute des attributs de variantes (ex. Couleur → Rouge, Taille → M) via le <code>VariantTreeService</code>. 5. Pour chaque variante feuille, elle renseigne un SKU, un prix spécifique et une quantité en stock. 6. Elle enregistre le produit, désormais visible sur la marketplace.
Scénario d'erreur	1. Le service de traduction automatique échoue. 2. Le système conserve le contenu saisi manuellement et signale l'échec de l'auto-remplissage.
Pré-conditions	La boutique est inscrite, validée et authentifiée.
Post-condition	Le produit et son arbre de variantes sont enregistrés et disponibles à la vente.

TABLE 3.2 – Description du cas d'utilisation « Gérer les produits et leurs variantes »

Description du cas d'utilisation : passer une commande

Cas d'utilisation	Passer une commande
Résumé	Ce cas permet au client de valider le contenu de son panier, de choisir une adresse de livraison et de confirmer sa commande.
Acteurs	Client
Scénario nominal	1. Le client s'authentifie et accède à son panier. 2. Il vérifie les produits, quantités et variantes sélectionnées. 3. Il choisit son gouvernorat de livraison, résolu en zone via <code>GeoZone::zoneForGovernorate()</code>. 4. Le système calcule le montant total, incluant la commission appliquée sur le prix de base de la boutique. 5. Le client confirme la commande. 6. Le système crée la commande (<code>orders</code>) et ses lignes (<code>order_items</code>) en conservant un instantané du nom et des données de la variante choisie.
Scénario d'erreur	1. Un produit ou une variante du panier n'est plus disponible en stock. 2. Le système signale l'indisponibilité et invite le client à ajuster son panier.
Pré-conditions	Le client est authentifié. Le panier contient au moins un produit disponible.
Post-condition	La commande est créée avec le statut <code>PENDING</code> et apparaît dans le suivi des commandes du client, de la boutique et de l'administrateur.

TABLE 3.3 – Description du cas d'utilisation « Passer une commande »

Description du cas d'utilisation : gérer les utilisateurs et les boutiques

Cas d'utilisation	Gérer les utilisateurs et les boutiques
Résumé	Ce cas permet à l'administrateur de consulter la liste des utilisateurs et des boutiques, de bloquer un compte ou d'activer/désactiver une boutique.
Acteurs	Administrateur
Scénario nominal	1. L'administrateur s'authentifie et accède à son tableau de bord. 2. Il consulte la liste des utilisateurs, filtrable par rôle (client, boutique, influenceur, société de livraison). 3. Il sélectionne un compte et applique une action (blocage, déblocage, validation). 4. Pour une boutique, il peut également consulter son taux de commission et son gouvernorat/zone de livraison. 5. Le système enregistre l'action et met à jour le statut du compte concerné.
Pré-conditions	L'administrateur est authentifié avec le rôle ADMIN.
Post-condition	Le statut de l'utilisateur ou de la boutique est mis à jour dans le système.

TABLE 3.4 – Description du cas d'utilisation « Gérer les utilisateurs et les boutiques »

3.4 Étude de quelques diagrammes des séquences

3.4.1 Diagramme de séquence : recherche sémantique de produits

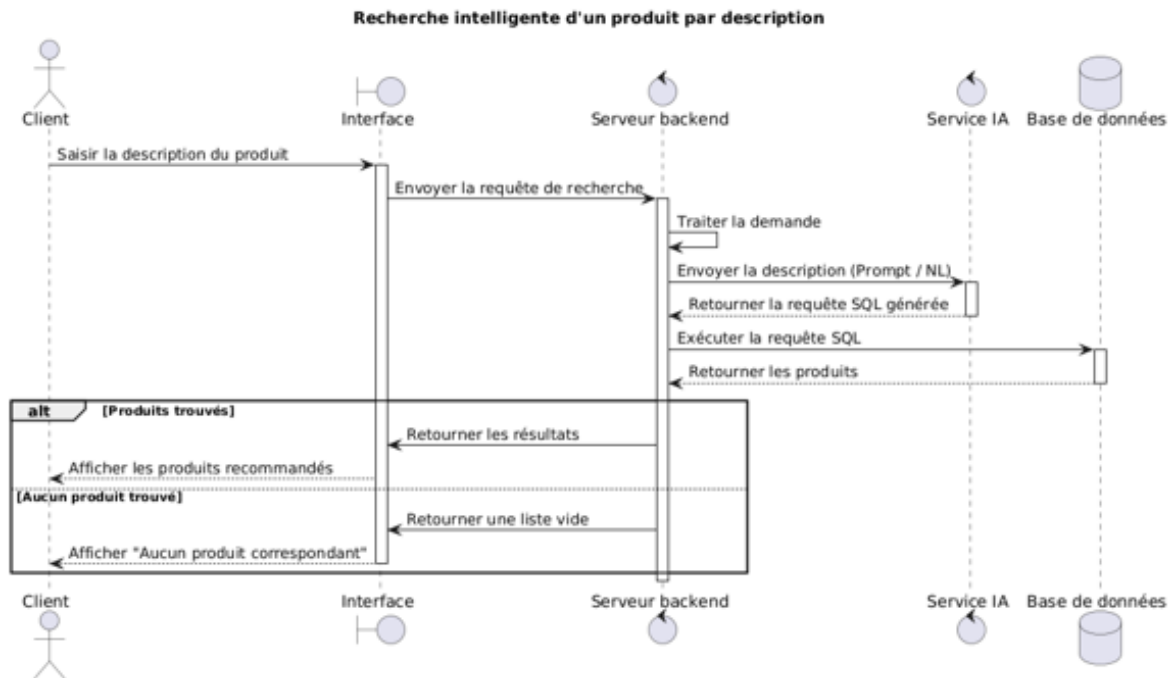


FIGURE 3.5 – Diagramme de séquence de la recherche sémantique de produits

Ce diagramme de séquence décrit le processus **RechercherProduitIA()**. Le client saisit sa requête dans la vue React, qui la transmet au contrôleur du **PublicController**. Ce dernier délègue le traitement au **ProductSemanticSearchService**, qui appelle le **GeminiEmbeddingService** pour vectoriser la requête, compare le vecteur obtenu aux embeddings déjà stockés dans **product_search_embeddings**, puis renvoie la liste des produits triés par similarité; en cas d'indisponibilité du service Gemini, le système retombe sur une recherche classique par mots-clés.

3.4.2 Diagramme de séquence : passer une commande

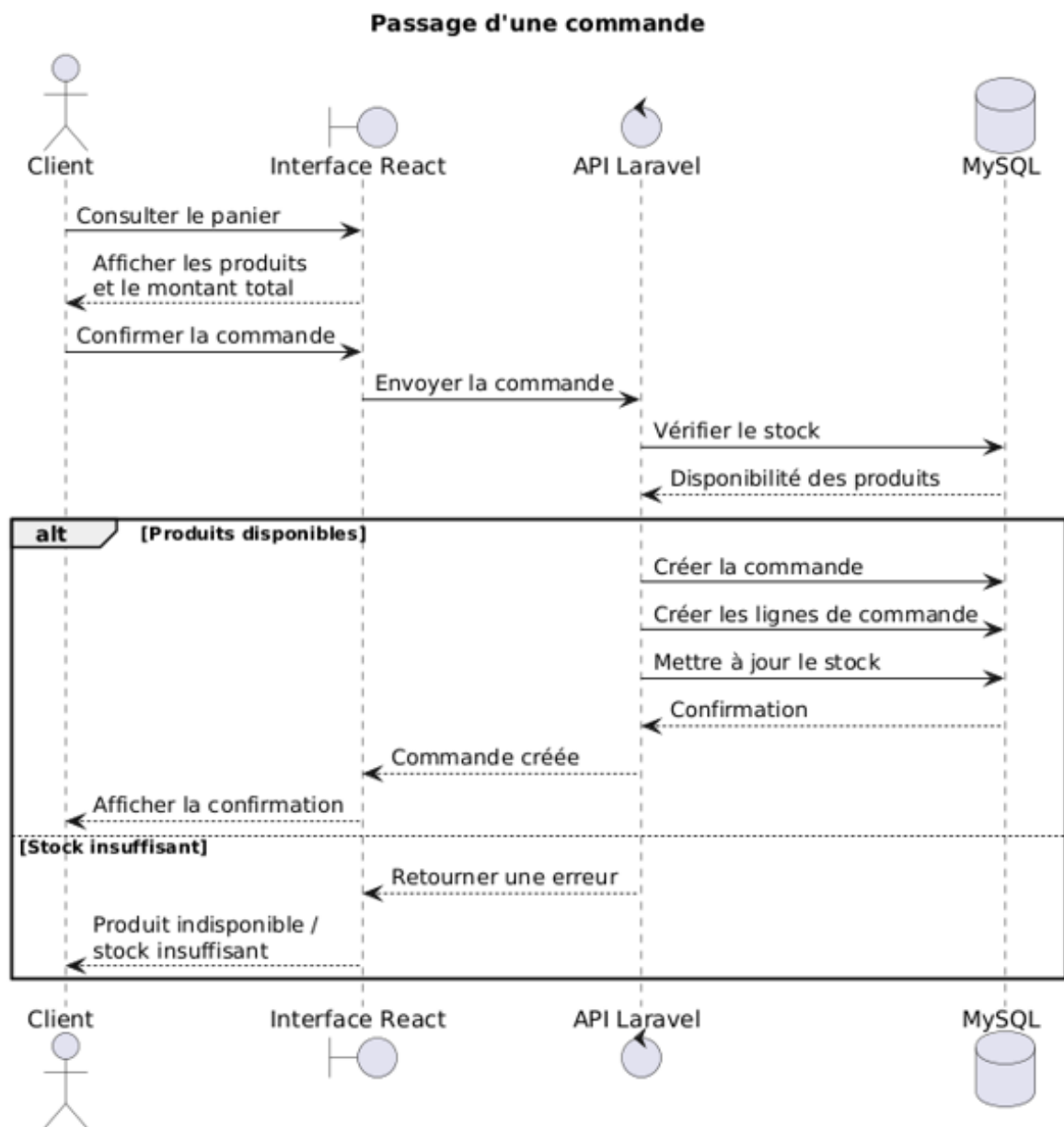


FIGURE 3.6 – Diagramme de séquence du passage de commande

Ce diagramme de séquence décrit le processus **PasserCommande()**. Le client valide son panier depuis la vue React, qui transmet les lignes du panier au contrôleur ; celui-ci vérifie la disponibilité du stock pour chaque variante, calcule le montant total en appliquant la commission de la plateforme, résout la zone de livraison à partir du gouvernorat de la boutique, puis crée la commande et ses lignes ; si un produit devient indisponible entre-temps, le contrôleur renvoie une erreur invitant le client à ajuster son panier.

3.4.3 Diagramme de séquence : modification du statut d'une commande

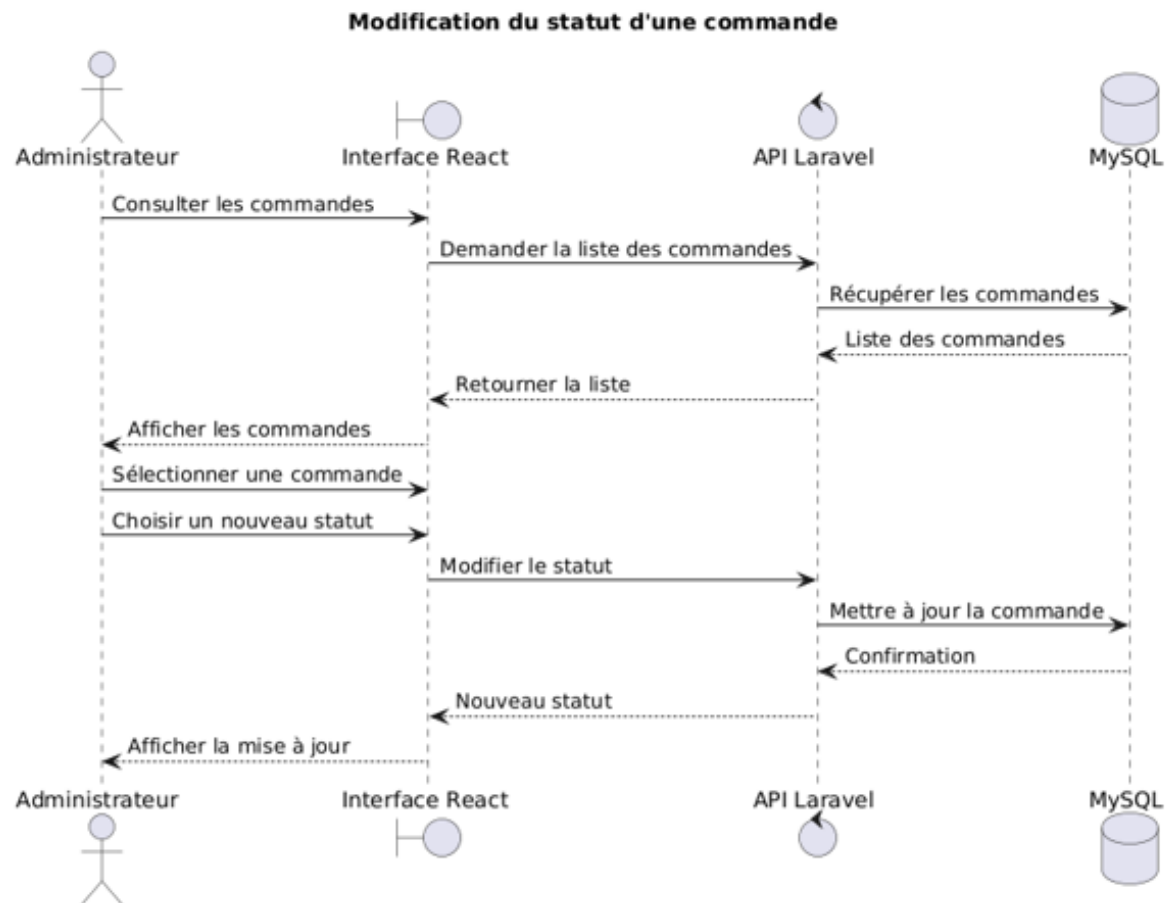


FIGURE 3.7 – Diagramme de séquence de la modification du statut d'une commande

Le diagramme de séquence de la **modification du statut d'une commande** présente le processus permettant à un administrateur de mettre à jour l'état d'une commande au sein de la plateforme.

Après authentification, l'administrateur accède à la liste des commandes et sélectionne la commande qu'il souhaite traiter. L'interface React transmet alors la demande au back-end Laravel à travers l'API.

Laravel récupère les informations de la commande depuis la base de données MySQL et les transmet à l'interface afin que l'administrateur puisse consulter les informations nécessaires.

L'administrateur choisit ensuite le nouveau statut de la commande. L'interface transmet cette modification à l'API Laravel. Le back-end vérifie la demande puis met à jour le statut correspondant dans la base de données MySQL.

Une fois la modification effectuée, la confirmation est retournée à l'interface React, qui affiche le nouveau statut à l'administrateur.

Le processus peut ainsi être résumé comme suit :

**Administrateur → Interface React → API Laravel → Base de données
MySQL → API Laravel → Interface React → Administrateur**

Ce processus permet ainsi à l'administrateur de suivre et de mettre à jour l'état des commandes de manière centralisée, tout en assurant la persistance des modifications dans la base de données.

3.4.4 Diagramme de séquence : reformulation d'un produit par IA

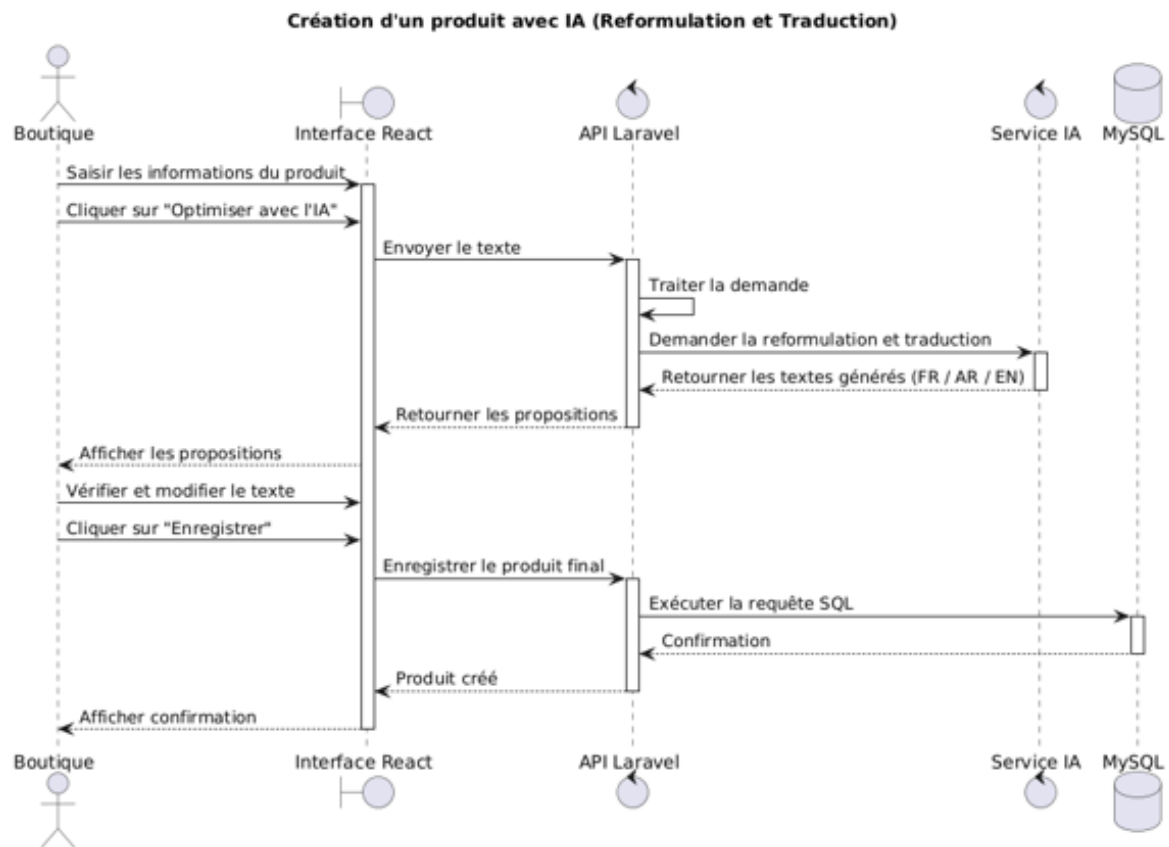


FIGURE 3.8 – Diagramme de séquence de la reformulation d'un produit par IA

Le diagramme de séquence de la **reformulation d'un produit par IA** présente le processus permettant à une boutique d'utiliser l'intelligence artificielle lors de la création d'une fiche produit.

La boutique accède au formulaire d'ajout d'un produit et renseigne les informations principales. Lorsqu'elle souhaite améliorer la description saisie, elle demande au système de la reformuler à l'aide de l'intelligence artificielle.

L'interface React transmet alors le texte au back-end Laravel à travers l'API. Laravel communique avec le service d'intelligence artificielle, qui analyse le contenu fourni et génère une nouvelle formulation.

La proposition générée est ensuite retournée à l'interface afin que la boutique puisse la consulter. Celle-ci conserve le contrôle du contenu et peut accepter la proposition ou la modifier avant de poursuivre l'enregistrement du produit.

Le processus peut ainsi être résumé comme suit :

**Boutique → Interface React → API Laravel → Service IA → API Laravel
→ Interface React → Boutique**

L'utilisation de l'intelligence artificielle permet ainsi d'assister la boutique dans la rédaction des fiches produits tout en conservant une validation humaine avant la publication du contenu.

3.5 Diagramme de classes

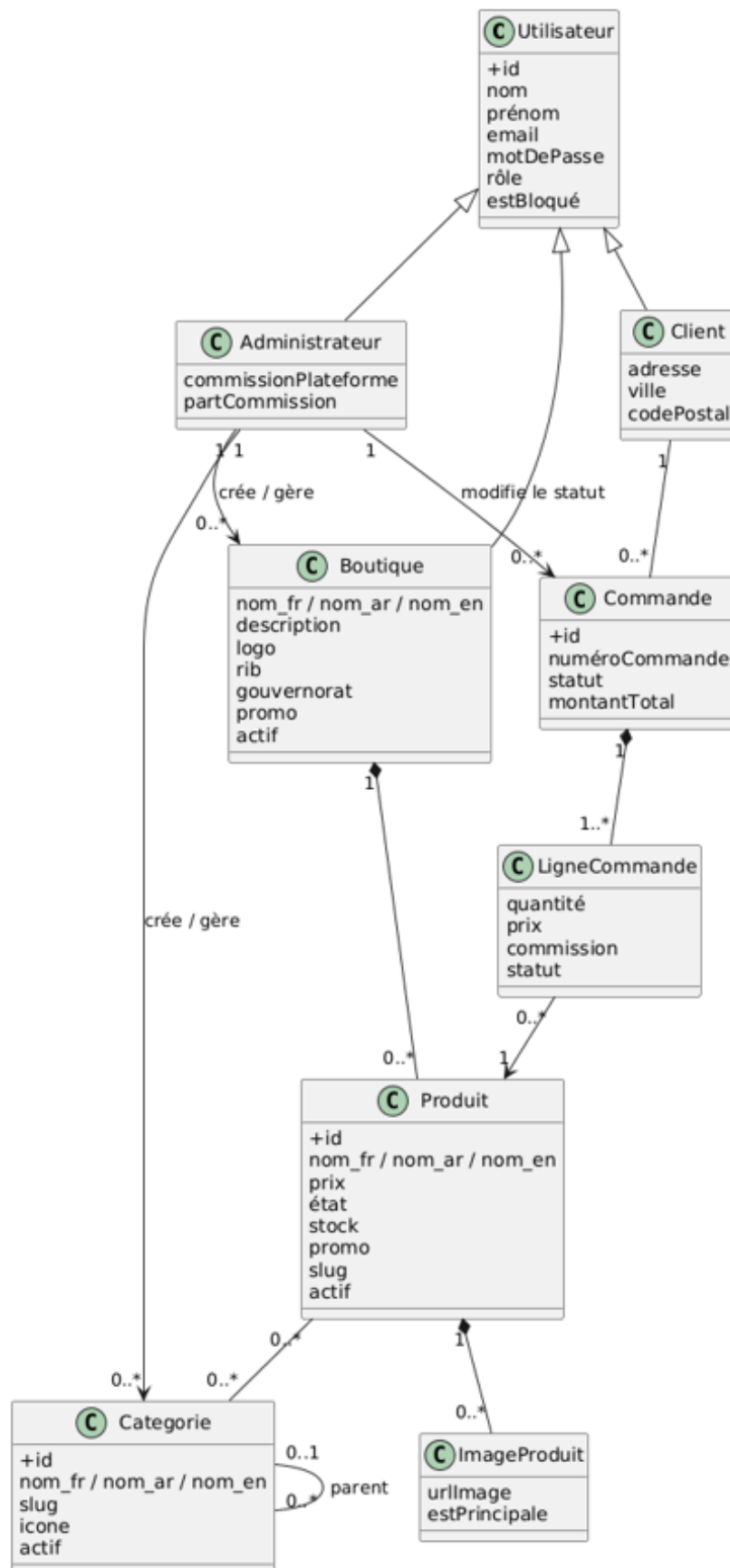


FIGURE 3.9 – Diagramme de classes

Le diagramme de classes présenté est un schéma statique UML qui décrit l'architecture du domaine de notre marketplace sociale Souk AI.

Au cœur du modèle, la super-classe **User** regroupe les attributs communs à tous les comptes (`id`, `nomComplet`, `email`, `motDePasse`, `role`) et se spécialise, via une relation 1–1, en plusieurs profils métier :

- **Admin**, chargé de la supervision globale de la plateforme et disposant d'un taux de commission (`commissionRate`) appliqué aux transactions ;
- **Store** (boutique), avec ses propres champs (`nomBoutique`, `gouvernorat`, `description`) et la capacité de publier des **Product** (`+ajouter(produit)`) ou de consulter ses commandes ;
- **Client**, qui peut passer des **Order** (`0..*`), noter des produits (`Rating`) et déposer des **Comment** ;
- **Influencer**, disposant d'un taux de commission propre (`influencers.commissionRate`) perçu sur les ventes issues de son référencement ;
- **ShippingCompany** et **ShippingEmp** (chauffeur), responsables de l'acheminement des commandes, un **ShippingEmp** pouvant être assigné à une commande via `driver_id`.

La classe **Product** est rattachée à une ou plusieurs **Category** (arbre auto-référencé via `category_product`) et possède un contenu trilingue (suffixes `_fr/`/`_ar/`/`_en`). Elle se décline en un arbre récursif de **ProductVariant** (attribut/valeur, ex. Couleur → Rouge), reliés entre eux par la table pivot `variant_links` pour former des combinaisons complexes ; chaque variante feuille porte un `sku`, un `price_override` et un `stock_quantity` propres. Les images sont rattachées aux produits ou aux variantes via **ProductAlbum** et la table pivot `variant_images`. La recherche sémantique s'appuie sur **ProductSearchEmbedding**, en relation 1–1 avec **Product**.

La classe **Order** regroupe un ensemble d'**OrderItem**, chacun conservant un instantané (`variant_name`, `variant_data`) de la variante commandée afin que la commande reste lisible même après suppression de la variante d'origine. L'évolution du statut global de la commande (`evaluateStatus()`) est automatiquement déclenchée par les changements de statut de ses lignes. Chaque commande peut donner lieu à plusieurs **Facture** (types `STORE`, `INFLUENCER`, `ADMIN`, `SHIPPING`, `SHIPPING_EMP`), reflétant la répartition des montants entre les différents acteurs.

Enfin, la classe **GeoZone** regroupe les gouvernorats tunisiens en zones de livraison ; la zone d'une boutique est résolue dynamiquement à partir de son gouvernorat, sans contrainte de clé étrangère, via `GeoZone::zoneForGovernorate()`.

3.6 Conclusion du chapitre

Ce chapitre a présenté les outils (UML via StarUML), puis formalisé nos processus métier avec les diagrammes de cas d'utilisation, de classes et de séquence. Nous y avons défini le périmètre fonctionnel de Souk AI (visiteur, client, boutique, administrateur), exposé la structure statique du domaine – utilisateurs, produits, variantes récursives, commandes et zones géographiques – et illustré les scénarios clés comme la recherche sémantique de produits, l'ajout d'un produit avec traduction automatique et le passage de commande. Cette étude conceptuelle fournit une base claire pour aborder la conception détaillée. Le chapitre suivant sera consacré à la mise en œuvre de notre application.

Chapitre 4

Réalisation de l'application

Sommaire

Conclusion Générale

Bibliographie

- [1] « Unified Modeling Language (UML) - Object Management Group » <https://www.omg.org/spec/UML/> – date de la dernière consultation : 25/08/2026.
- [2] « StarUML - UML Modeling Tool » <https://staruml.io/> – date de la dernière consultation : 25/08/2026.
- [3] « Visual Studio Code - Documentation officielle » <https://code.visualstudio.com/docs> – date de la dernière consultation : 25/08/2026.
- [4] « Visual Studio Code - Getting Started » <https://code.visualstudio.com/docs/getstarted/overview> – date de la dernière consultation : 25/08/2026.
- [5] « Laravel - The PHP Framework for Web Artisans » <https://laravel.com/> – date de la dernière consultation : 25/08/2026.
- [6] « Laravel - Documentation officielle » <https://laravel.com/docs> – date de la dernière consultation : 25/08/2026.
- [7] « Laravel - Routing » <https://laravel.com/docs/routing> – date de la dernière consultation : 25/08/2026.
- [8] « Laravel - Eloquent ORM » <https://laravel.com/docs/eloquent> – date de la dernière consultation : 25/08/2026.
- [9] « Laravel - Controllers » <https://laravel.com/docs/controllers> – date de la dernière consultation : 25/08/2026.
- [10] « React - Documentation officielle » <https://react.dev/> – date de la dernière consultation : 25/08/2026.
- [11] « React - Learn » <https://react.dev/learn> – date de la dernière consultation : 25/08/2026.
- [12] « React - API Reference » <https://react.dev/reference/react> – date de la dernière consultation : 25/08/2026.
- [13] « Tailwind CSS - Documentation officielle » <https://tailwindcss.com/docs> – date de la dernière consultation : 25/08/2026.
- [14] « Tailwind CSS - Utility-First CSS Framework » <https://tailwindcss.com/> – date de la dernière consultation : 25/08/2026.
- [15] « MySQL - Documentation officielle » <https://dev.mysql.com/doc/> – date de la dernière consultation : 25/08/2026.

- [16] « MySQL 8.0 Reference Manual » <https://dev.mysql.com/doc/refman/8.0/en/> – date de la dernière consultation : 25/08/2026.
- [17] « Docker - Documentation officielle » <https://docs.docker.com/> – date de la dernière consultation : 25/08/2026.
- [18] « Docker - Get Started » <https://docs.docker.com/get-started/> – date de la dernière consultation : 25/08/2026.
- [19] « Docker Compose - Documentation officielle » <https://docs.docker.com/compose/> – date de la dernière consultation : 25/08/2026.
- [20] « Postman - Documentation officielle » <https://learning.postman.com/docs/> – date de la dernière consultation : 25/08/2026.
- [21] « Postman - API Client » <https://www.postman.com/product/api-client/> – date de la dernière consultation : 25/08/2026.
- [22] « Postman - Sending API Requests » <https://learning.postman.com/docs/sending-requests/requests/> – date de la dernière consultation : 25/08/2026.
- [23] « REST - MDN Web Docs » <https://developer.mozilla.org/en-US/docs/Glossary/REST> – date de la dernière consultation : 25/08/2026.
- [24] « HTTP - MDN Web Docs » <https://developer.mozilla.org/en-US/docs/Web/HTTP> – date de la dernière consultation : 25/08/2026.